PYTHON → KUBERNETES

DEIN TOOLING COCKPIT

VON PYTHON BIS K8S

PYTHON - LIVE & DIREKT

Sandbox Python 3.9 mit echtem Code-Run hier im Chat. Keine Theorie, sofortige Ergebnisse.

Pandas
Numpy, SciKit, Matplotlib drin

Run
Direkt ausführen & debuggen

Export
.py .csv .xlsx .pdf

CORE STACK

DATA & ML

- pandas, numpy, scipy, statsmodels
- scikit-learn, OpenCV, geopandas
- matplotlib, plotly, altair

FILES & DOCS

- openpyxl, python-docx, python-pptx
- PyMuPDF, PyPDF2, reportlab
- Deine /mnt/data Dateien direkt lesen

WAS ICH MACHE

- Code schreiben + sofort ausführen
- CSV/Excel bereinigen, analysieren
- Charts bauen, Fehler zeigen
- Modulare Skripte, nicht nur Blöcke

WORKFLOW

- Upload → Analysieren → Visualisieren
- Forecast, Report, Automation
- Speichern als Download-Datei
- Iterativ debuggen im Chat

JS/TS - INTERAKTIVE VORSCHAUEN

Echte React/TS Apps, die sofort hier im Chat laufen – klickbar, ohne Setup.

LIVE PREVIEW
REACT + TSX
Keine externen Links

LIVE IM CHAT

REACT / TS WIRD GEBAUT

→ Automatische Preview – du klickst sofort
→ State, Hooks, Charts alles drin
→ Iterativ: „mach Button blau“ → live

0 Setup
Kein Vercel nötig

Insta Edit
Direktes Feedback

WAS ICH BAUE

→ Dashboards, Admin-Tools
→ Landingpages, Portfolios
→ Tools & Spiele

NODE BACKEND

→ package.json, Services, Routes
→ Tests + .env + README
→ Zum Download fertig

PLUS

→ Fotos & Grafiken einbaubar
→ Lint + Performance Tipps

WORKFLOW - SO SCHNELL

IDEA → PREVIEW IN SEKUNDEN

1. Prompt
„Dashboard für API X“

2. Build
TSX + CSS live

3. Iterate
„Füg Filter hinzu“

GO, C++, RUST - READY-TO-BUILD

KEIN LIVE-COMPILER HIER,
ABER 100% BUILD-FÄHIGE PROJEKTE

GO

GO.MOD + MAKEFILE

1. **Idiomatisch:** Module, Concurrency mit goroutines/channels
2. **Tests:** go test mit Race-Detector, Table-Driven
3. **Ready:** main.go, handlers, Dockerfile direkt lauffähig

C/C++

CMAKE • VCPKG

1. **C11-C23, C++17/20/23:** moderne Standards
2. **Safety:** Memory-Leaks, Pointer, UBSan Hinweise
3. **Projekt:** src/, CMakeLists.txt, vcpkg.json

RUST

CARGO • ASYNC

1. **Cargo.toml:** Traits, Lifetimes, Ownership erklärt
2. **Async:** tokio, clippy-Tipps, borrow checker Help
3. **Projekt:** src/main.rs + lib.rs, Tests, CI-ready

↗ **Workflow:** Code schreiben → als Datei ablegen → du baust lokal mit go build / cargo run / cmake

Best Practice: Kommentare + Error-Handling idiomatisch, nicht nur Snippets

SHELL & SQL - AUTOMATION & DATEN

Robuste Skripte, saubere Queries – mit Best Practices direkt produktionsreif. SQLite teste ich sogar live hier.

01 BASH

SET -EUO PIPEFAIL

- Sicheres Error-Handling, Logging
- Argument-Parsing & Validierung
- Logik via Python simuliert, final bei dir

02 POWERSHELL

CMDLETS & MODULE

- Cross-Platform Checks
- Module-Struktur, Error-Action
- Jede Zeile erklärt – super zum Lernen

03 POSTGRES / SQLITE

QUERIES + TUNING

- Komplexe JOINs, Index-Vorschläge
- Execution Plan verstehen
- SQLite live in Python Sandbox testen

🔒 **Safe by default** ⚡ **Automatisierbar** 📦 **Download als .sh / .sql**

Bash · PowerShell · PostgreSQL · MySQL · SQLite · MSSQL · BigQuery

BEST PRACTICES PRODUCTION READY · SECURE

DOCKER - SCHLANK & SICHER

1. **Multi-Stage Builds**
   Builder + Runtime getrennt. Nur Binary ins Final-Image → 10x kleiner.

2. **Non-root & Distroless**
   USER 1001, alpine / gcr.io/distroless. Weniger CVEs, weniger Attack Surface.

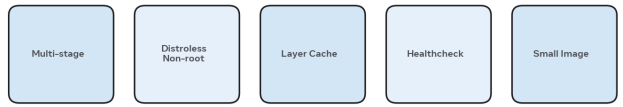
3. **Layer-Cache optimal**
   Dependencies zuerst kopieren, COPY --link, BuildKit Cache Mounts.

4. **.dockerignore + Healthcheck**
   Kein .git/node_modules. HEALTHCHECK + dumb-init für sauberes Shutdown.

< 50MB
Ziel für Go/Rust Images mit Distroless

Non-root
Pflicht in jedem Dockerfile

Cache
Builds in Sekunden, nicht Minuten

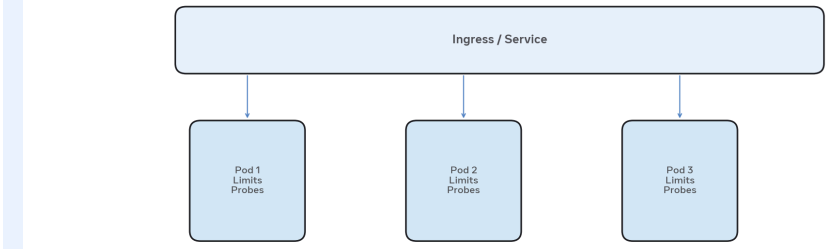


MULTI-STAGE → DISTROLESS → LAYER CACHE → HEALTHCHECK → SMALL IMAGE

KUBERNETES – ROBUST DEPLOYN

1. **Deployment, Service, Ingress, ConfigMap/Secret** – sauber getrennt, keine Monster-YAMLs
2. **Resource Limits + Liveness/Readiness Probes** – stabil, keine OOM-Kills
3. **Rolling Updates, HPA** – Zero-Downtime & Auto-Scaling
4. **Kustomize / Helm** für Environments – dev → staging → prod
5. Security: non-root, readOnlyFS, NetworkPolicies, RBAC

PRODUCTION READY | 12-FACTOR | GITOPS READY



INGRESS / SERVICE → PODS MIT LIMITS & PROBES

HPA

DEPLOYMENT

replicas, strategy rollingUpdate, anti-affinity

SERVICE + INGRESS

ClusterIP, path-based routing, TLS

CONFIG/SECRET

Getrennt, kein Hardcode, envFrom

OBSERVABILITY

Probes, limits, labels für Monitoring

TERRAFORM - INFRASTRUKTUR ALS CODE

Saubere Module, reproduzierbare Infrastruktur. Ich generiere Dir production-ready HCL – nicht nur Snippets.

HCL + AWS/GCP

Module • State • Provider
Validate before Apply

STRUCTURE

MODULE SAUBER GETRENNT

- variables.tf / outputs.tf / main.tf getrennt
- Wiederverwendbare Module mit Beispiel
- Keine Hardcodes, alles variabel

```
module "vpc" {
  source = "./modules/vpc"
  env = var.env
}
```

REMOTE STATE + LOCKING

- S3 + DynamoDB oder GCS + Lock
- Kein lokaler state.tfstate im Git
- State-Migration sauber dokumentiert

```
terraform {
 backend "s3" {
  encrypt = true
 }
}
```

PROVIDER-PINNING

- ~> Versionen in required_providers
- Terraform Version gepinnt
- Reproduzierbare Builds garantiert

```
required_providers {
 aws = {
  version = "~> 5.0"
 }
}
```

VALIDATE & PLAN

- tflint + tfsec + validate im CI
- terraform plan vor jedem apply
- Outputs für K8s / App direkt nutzbar

```
terraform validate
terraform plan -out plan.tfplan
```

GITHUB ACTIONS

GO • DOCKER • TRIVY

CI/CD PIPELINE

GO BEISPIEL

1 **Trigger auf push / PR**
on: [push, pull_request] → main + feature branches

2 **golangci-lint + go test -race**
Race-Detector, Coverage, vet – fail fast vor Build

3 **Docker Build + Push mit Cache**
buildx, GHA cache, layer caching → ghcr.io

4 **Trivy Security Scan + Deploy**
Vulnerability scan, danach K8s deploy / argocd sync

```
# .github/workflows/go.yml
jobs:
  lint: golangci-lint-action@v6
  test: go test -race -cover ./...
  docker: build-push-action + cache-from: gha
```



LINT → TEST → BUILD → SCAN → DEPLOY

PRODUCTION

✓ **Cache**
gha + registry cache = 70% schneller

✓ **Security**
Trivy blockt HIGH/CRITICAL CVEs

✓ **go.mod**
dependencies via go.sum gepinnt

✓ **Push**
ghcr.io + semantic version Tags

GITHUB ACTIONS

RUST • CARGO • CLIPPY • DOCKER

CI/CD PIPELINE
RUST EXAMPLE

1 **cargo fmt --check, clippy -- -D warnings**
Schon im PR: Format + Lint fail → kein Merge. Spart Review-Zeit.

2 **cargo test + nextest**
Unit + Integration, mit Caching für Cargo Registry. Schnell, stabil.

3 **Cross-Build & Docker Build**
x86_64 + ARM64, Multi-Stage, Push mit Buildx Cache.

4 **Caching & Security**
Cargo cache, Trivy Scan, Cosign. Kein Secret im Image.

fmt + clippy | cargo nextest | cross-build | trivy scan



LINT → TEST → BUILD DOCKER → SCAN → DEPLOY K8S

RUST

```
jobs:
  lint: cargo fmt --check && clippy
  test: cargo nextest run --locked
  build: docker/build-push-action with cache
  scan: aquasecurity/trivy-action
```

DEIN WIEDERVERWENDBARER DEV-SKILL

Aus diesem Chat-Verlauf wird ein reusable Skill.
Jedes neue Projekt startet in Sekunden.

01 NAME

FULLSTACK-PROJECT-BOOTSTRAPPER

Skill speichert unseren kompletten Workflow als wiederverwendbare Fähigkeit.

```
/skill install
fullstack-bootstrapper
```

02 INPUT

WAS DU GIBST

- Sprache: Go / Rust / Python / TS
- Stack: Docker + K8s + CI + DB
- Ziel: z.B. „Microservice mit API“

```
Stack: Go + Docker
+ K8s + GitHub Actions
```

03 OUTPUT

WAS DU BEKOMMST

- Projektstruktur + README + .gitignore
- Dockerfile multi-stage, non-root
- K8s Manifeste + Terraform Modul
- CI YAML (lint, test, build, Trivy)

♻️ 100% wiederverwendbar

⚡ Bootstrapt in < 30s jedes neue Repo

📦 Kein Copy-Paste mehr, echter Skill

INPUT → SKILL → OUTPUT = PRODUCTION READY

SO STARTEN WIR

1

SPRACHE & ZIEL WÄHLEN

Go, Rust, Python, JS/TS – was brauchst du wirklich?

2

ICH GENERIERE SKELETT

Projektstruktur, go.mod / Cargo.toml, Makefile, Tests, README

3

CI/CD + DOCKER + K8S

Dockerfile, K8s-Manifeste, GitHub Actions direkt dabei

4

DU BAUST, ICH DEBUGGE

Lokal bauen, ich helfe live bei Fehlern & Optimierung

→ BEREIT? SAG STACK: z.B. “Go + Docker + K8s + CI”

GIT-READY · DOWNLOAD · PRODUCTION

Full-Stack Tooling Cockpit – von Python bis Kubernetes, wiederverwendbar für jedes neue Projekt